Test-Style Solution

VNUHCM–University of Science
Course: Computer Hardware (ECE341)

Remark for Question 3: the statement on Page 3 appears slightly inconsistent with the data on Page 2. To keep the solution coherent, the standard interpretation is used:

$$\texttt{Load R4, 200(R3)} \Rightarrow R4 \leftarrow M[R3 + 200].$$

With $R3 = 28000$, this means address 28200, hence $R4 \leftarrow 500$.

Question 1 (25 points)

Three processors execute the same ISA:

$$\begin{aligned} P1 &: f_1 = 3 \text{ GHz}, \quad \text{CPI}_1 = 2.5, \\ P2 &: f_2 = 2.5 \text{ GHz}, \quad \text{CPI}_2 = 1.0, \\ P3 &: f_3 = 4.0 \text{ GHz}, \quad \text{CPI}_3 = 2.2. \end{aligned}$$

Recall:

$$\text{IPS} = \frac{\text{Clock rate}}{\text{CPI}}.$$

(a) Highest performance in instructions/second

$$\text{IPS}_1 = \frac{3 \times 10^9}{2.5} = 1.2 \times 10^9$$

$$\text{IPS}_2 = \frac{2.5 \times 10^9}{1.0} = 2.5 \times 10^9$$

$$\text{IPS}_3 = \frac{4 \times 10^9}{2.2} \approx 1.818 \times 10^9$$

Therefore,

$$\boxed{P2 \text{ has the highest performance}}$$

with

$$\boxed{2.5 \times 10^9 \text{ instructions/s}}.$$

(b) If each processor executes for 15 seconds, find cycles and instructions

Use

$$\text{Cycles} = f \cdot T, \qquad \text{Instructions} = \frac{\text{Cycles}}{\text{CPI}}.$$

Processor $P1$:

$$\text{Cycles}_1 = 3 \times 10^9 \times 15 = 45 \times 10^9$$

$$\text{Instructions}_1 = \frac{45 \times 10^9}{2.5} = 18 \times 10^9$$

Processor $P2$:

$$\text{Cycles}_2 = 2.5 \times 10^9 \times 15 = 37.5 \times 10^9$$

$$\text{Instructions}_2 = \frac{37.5 \times 10^9}{1.0} = 37.5 \times 10^9$$

Processor $P3$:

$$\text{Cycles}_3 = 4 \times 10^9 \times 15 = 60 \times 10^9$$

$$\text{Instructions}_3 = \frac{60 \times 10^9}{2.2} \approx 27.273 \times 10^9$$

Hence:

Processor	Cycles	Instructions
$P1$	45×10^9	18×10^9
$P2$	37.5×10^9	37.5×10^9
$P3$	60×10^9	27.273×10^9

(c) Reduce execution time by 40% while CPI increases by 30%

Original time:

$$T = \frac{IC \cdot CPI}{f}$$

New constraints:

$$T' = 0.6T, \qquad CPI' = 1.3\,CPI$$

Then

$$T' = \frac{IC \cdot CPI'}{f'} = \frac{IC \cdot 1.3\,CPI}{f'}$$

and also

$$T' = 0.6\frac{IC \cdot CPI}{f}$$

Equating:

$$\frac{1.3}{f'} = \frac{0.6}{f} \quad \Rightarrow \quad f' = \frac{1.3}{0.6}f = \frac{13}{6}f \approx 2.1667f$$

So each processor would require:

$$f_1' = 2.1667 \times 3 = 6.5 \text{ GHz}$$

$$f_2' = 2.1667 \times 2.5 \approx 5.417 \text{ GHz}$$

$$f_3' = 2.1667 \times 4 \approx 8.667 \text{ GHz}$$

Therefore,

$$\boxed{f' = 2.1667f}$$

and specifically:

$$\boxed{P1 : 6.5 \text{ GHz}, \quad P2 : 5.417 \text{ GHz}, \quad P3 : 8.667 \text{ GHz}}$$

Question 2 (25 points)

Question 2-1

A 5-stage pipelined RISC processor $C1$ runs at 2.6 GHz.

Instruction mix:

$$\text{Branches} = 10\%, \quad \text{Loads} = 20\%, \quad \text{Stores} = 20\%, \quad \text{Arithmetic} = 50\%$$

Other information:

- No data dependencies.
- Branch predictor accuracy = 85%.
- Branch outcome resolved in stage 3.
- 100% instruction fetches hit cache.
- 100% data writes hit cache.
- 98% data reads hit cache.
- Total instruction count = 20 billion.
- Required execution time < 10 seconds.

Let the main-memory miss penalty be P cycles.

Step 1: Ideal CPI

For a perfect 5-stage pipeline without stalls,

$$CPI_{\text{ideal}} = 1.$$

Step 2: Branch penalty contribution

Misprediction rate:

$$1 - 0.85 = 0.15$$

Branches are 10% of instructions, so mispredicted branches are:

$$0.10 \times 0.15 = 0.015$$

Since the actual outcome is known in stage 3, a wrong prediction flushes the instructions in stages 1 and 2. Thus, the penalty is taken as 2 cycles per misprediction.

So branch CPI overhead is:

$$\Delta CPI_{\text{branch}} = 0.015 \times 2 = 0.03$$

Step 3: Load-miss penalty contribution

Loads are 20% of all instructions.

Load miss rate:

$$1 - 0.98 = 0.02$$

Thus the fraction of instructions causing a load miss is:

$$0.20 \times 0.02 = 0.004$$

Each miss costs P cycles, hence:

$$\Delta CPI_{\text{mem}} = 0.004P$$

Step 4: Total CPI

$$CPI = 1 + 0.03 + 0.004P = 1.03 + 0.004P$$

Step 5: Time constraint

Execution time:

$$T = \frac{IC \cdot CPI}{f}$$

Substitute:

$$10 \geq \frac{20 \times 10^9 \cdot CPI}{2.6 \times 10^9}$$

So

$$CPI \leq \frac{10 \cdot 2.6 \times 10^9}{20 \times 10^9} = 1.3$$

Thus:

$$1.03 + 0.004P \leq 1.3$$

$$0.004P \leq 0.27$$

$$P \leq \frac{0.27}{0.004} = 67.5$$

Hence the upper bound is

$$\boxed{P_{\max} = 67.5 \text{ cycles}}$$

If an integer number of cycles is required:

$$\boxed{P_{\max} = 67 \text{ cycles}}$$

Question 2-2

Design a Mealy FSM that detects serial input patterns:

- output $z = 10$ when data $= 1101$,
- output $z = 01$ when data $= 1110$,
- otherwise $z = 00$,

including overlap.

State definition

Use states corresponding to the longest suffix that is also a prefix of one of the target patterns.

State	Meaning
S_0	ϵ (no useful prefix matched)
S_1	1
S_{11}	11
S_{110}	110
S_{111}	111

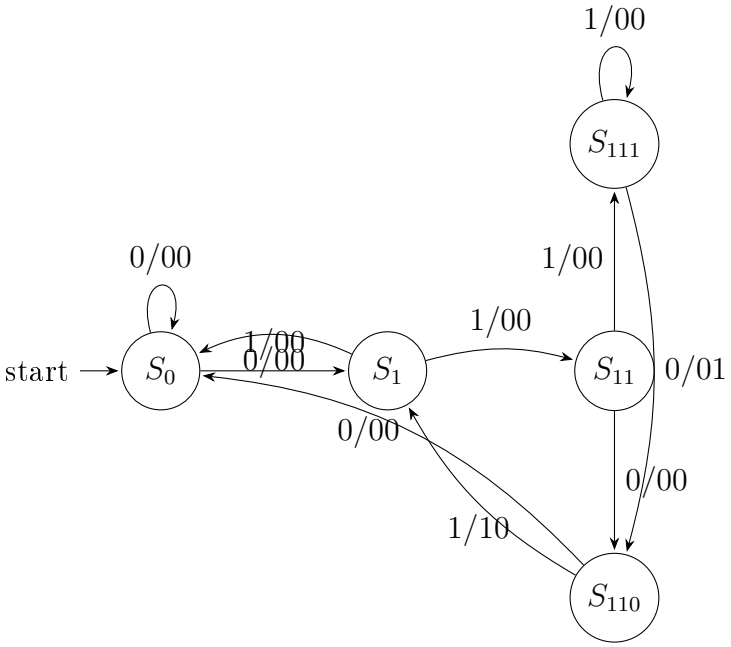
State transition/output table

Present state	$x = 0$	$x = 1$
S_0	$S_0/00$	$S_1/00$
S_1	$S_0/00$	$S_{11}/00$
S_{11}	$S_{110}/00$	$S_{111}/00$
S_{110}	$S_0/00$	$S_1/10$
S_{111}	$S_{110}/01$	$S_{111}/00$

Explanation of overlap handling

- From S_{110}, if the next bit is 1, we detect 1101, so output is 10. The ending suffix is 1, so the next state is S_1.
- From S_{111}, if the next bit is 0, we detect 1110, so output is 01. The ending suffix is 110, so the next state is S_{110}.

Optional state diagram (TikZ)



Question 3 (25 points)

Instruction sequence:

$$I_1 : \texttt{Load R4, 200(R3)}$$
$$I_2 : \texttt{Add R5, R2, R4}$$
$$I_3 : \texttt{Store R5, 400(R3)}$$

Given:

$$R2 = 200, \quad R3 = 28000, \quad M[28000] = 400, \quad M[28200] = 500, \quad M[28400] = 600$$

Using the standard addressing interpretation:

$$I_1 : \quad R4 \leftarrow M[R3 + 200] = M[28200] = 500$$

Then:

$$I_2 : \quad R5 \leftarrow R2 + R4 = 200 + 500 = 700$$

Finally:

$$I_3 : \quad M[R3 + 400] \leftarrow R5 = M[28400] \leftarrow 700$$

(a) For each instruction

Instruction 1: `Load R4, 200(R3)`

- **MuxB selection:** immediate offset 200, so

  $$\boxed{\text{MuxB selects input 1 (immediate)}}$$

- **Stage 4 (RY):** load obtains memory data from $M[28200] = 500$, hence

  $$\boxed{RY = 500}$$

- **Stage 5 (RF_write):** load writes back into $R4$, so

  $$\boxed{RF\_write = 1}$$

Instruction 2: `Add R5, R2, R4`

- **MuxB selection:** second ALU operand comes from register $R4$, so

  $$\boxed{\text{MuxB selects input 0 (register operand)}}$$

- **Stage 4 (RY):** ALU result is forwarded:

  $$RY = R2 + R4 = 200 + 500 = 700$$

  therefore

  $$\boxed{RY = 700}$$

- **Stage 5 (RF_write):** add writes result into $R5$, so

  $$\boxed{RF\_write = 1}$$

Instruction 3: `Store R5, 400(R3)`

- **MuxB selection:** effective address uses immediate offset 400, so

$$\boxed{\text{MuxB selects input 1 (immediate)}}$$

- **Stage 4 (RY):** the effective address is

$$R3 + 400 = 28000 + 400 = 28400$$

  In the common 5-stage datapath convention, RY receives the pass-through ALU/address value in stage 4 for a store:

$$\boxed{RY = 28400}$$

- **Stage 5 (RF_write):** store does not write a register, so

$$\boxed{RF\_write = 0}$$

(b) Final contents of memory locations 28400 and 28800

Only the store updates memory:

$$M[28400] \leftarrow 700$$

No instruction accesses address 28800, so it remains unchanged.

Thus:

$$\boxed{M[28400] = 700}$$

$$\boxed{M[28800] \text{ unchanged}}$$

Question 4 (25 points)

Instruction sequence:

$$I_1 : \texttt{LOAD R4, \#200(R2)}$$
$$I_2 : \texttt{XOR R5, R3, R4}$$
$$I_3 : \texttt{CALL\_REGISTER R5}$$

Given:

$$R2 = 2000, \quad R3 = 60, \quad R4 = 100,$$
$$M[2000] = 10, \quad M[2200] = 26$$

Step 1: Execute instruction semantics

Instruction 1: `LOAD R4, #200(R2)`

Effective address:

$$R2 + 200 = 2000 + 200 = 2200$$

So:

$$R4 \leftarrow M[2200] = 26$$

Thus the inter-stage ALU register after stage 3 is:

$$\boxed{RZ_{(I_1)} = 2200}$$

Instruction 2: `XOR R5, R3, R4`

After I_1, $R4 = 26$. Hence:

$$R5 \leftarrow R3 \oplus R4 = 60 \oplus 26$$

In binary:

$$60 = 00111100_2, \qquad 26 = 00011010_2$$
$$60 \oplus 26 = 00100110_2 = 38$$

So:

$$\boxed{R5 = 38}$$

and after stage 3:

$$\boxed{RZ_{(I_2)} = 38}$$

Instruction 3: `CALL_REGISTER R5`

The call target is the value in $R5$, therefore the new PC becomes:

$$\boxed{PC = 38}$$

Control signals

From the datapath:

- Y_select chooses the value written into RY:
  - 0: ALU result / RZ
  - 1: Memory data
  - 2: Return address
- RF_write indicates whether a register is written in stage 5.
- C_select chooses the destination register:
  - 0: $IR_{26\text{-}22}$
  - 1: $IR_{21\text{-}17}$
  - 2: LINK

Instruction 1: `LOAD R4, #200(R2)`

- Load gets data from memory in stage 4:

$$\boxed{Y\_select = 1}$$

- Load writes destination register in stage 5:

$$\boxed{RF\_write = 1}$$

- For a load, the destination is the register field used as load destination:

$$\boxed{C\_select = 0}$$

Instruction 2: `XOR R5, R3, R4`

- Arithmetic/logic instructions forward ALU result:

$$\boxed{Y\_select = 0}$$

- XOR writes result into $R5$:

$$\boxed{RF\_write = 1}$$

- Destination register is the third field:

$$\boxed{C\_select = 1}$$

Instruction 3: `CALL_REGISTER R5`

- Call saves return address:

$$\boxed{Y\_select = 2}$$

- Call writes the return address into LINK:

$$\boxed{RF\_write = 1}$$

- Destination register is LINK:

$$\boxed{C\_select = 2}$$

Final summary table

Instruction	Y_select	RF_write	C_select
`LOAD R4,#200(R2)`	1	1	0
`XOR R5,R3,R4`	0	1	1
`CALL_REGISTER R5`	2	1	2

Also,

$$\boxed{RZ_{(I_1)} = 2200}$$

$$\boxed{RZ_{(I_2)} = 38}$$

$$\boxed{PC_{\text{final}} = 38}$$

End of Solution